

**Experiential Learning
Project Assessment-I
on
“Number Guessing Game”
under the course
Python Programming Language (N-PCCCM304P)**

Presented By:



**Under the Guidance
of**

**Session:2026-27 (ODD)
Department of CSE(AI&ML)**

**S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND
RESEARCH,NAGPUR**

An Autonomous Institute, Affiliated to RTMNU, Nagpur

Number Guessing Race

Experiential Learning Project Assessment-I
Python Programming Language (N-PCCCM304P)

S. B. Jain Institute of Technology Management and Research, Nagpur

Contents

1. Problem Statement
2. Objectives
3. Algorithm & Pseudocode
4. Understanding Python Logic
5. Game Flow & Working
6. GitHub & Collaboration
7. Individual Contribution & Output

1. Problem Statement

Interactive Gaming Challenges

Standard text-based games lack dynamic interaction between human players and computer opponents, missing real-time competitive elements and robust validation.

The Solution: Number Guessing Race

A turn-based interactive race where you compete head-to-head against the computer to guess a secret number first, utilizing loops, conditionals, and input validation.

2. Objectives

- ✓ **Turn-Based Competition:** Implement a competitive gameplay loop between user and computer.
- ✓ **Control Flow:** Master practical application of `while` and `for` loops.
- ✓ **Conditional Logic:** Utilize `if/elif/else` branches to give high/low hints.
- ✓ **Input Validation:** Ensure program resilience against invalid user entries.

3. Algorithm

Initialization

1. Start the game & import random module.
2. Generate a random secret number within a specified range (e.g., 1 to 100).
3. Initialize attempt counters for both player and computer.

Execution Loop

4. Accept validated user input guess.
5. Generate automated computer guess.
6. Compare guesses against secret number using conditional branches.
7. Declare winner upon matching the secret number.

4. Understanding Python Logic

```
# Secret Number Generation & Loop
import random
secret = random.randint(1, 100)
running = True

while running:
    guess = int(input("Enter guess: "))
    if guess == secret:
        print("You Win!")
        break
    elif guess < secret:
        print("Too low!")
    else:
        print("Too high!")
```

Core Constructs

Loops: Controls continuous turns until a match occurs.

Conditionals: Evaluates relationship between guess and secret number.

Validation: Catches non-numeric inputs gracefully.

5. Game Flow & Working

Player Turn

User inputs a guess. The program validates input type, compares against the secret target, and outputs feedback ("Too High" or "Too Low").

Computer Turn

Computer uses smart boundary adjustments (binary search logic) to make optimal guesses, creating a genuine race atmosphere.

6. GitHub & Collaboration

Repository Management

- Created dedicated public repository: Number - Guessing - Race
- Organized folder structure and documented README instructions.

Version Control

- Meaningful commits for feature additions.
- Branching workflows and collaborative pull requests used effectively by team members.

7. Individual Contribution



Logic Design: Designed the turn-based state machine and validation functions.



Python Coding: Developed core gameplay loops and computer AI guessing heuristics.



Testing & Debugging: Verified edge cases like out-of-range inputs and string entries.



Documentation: Prepared presentation deck and repository documentation.

Code Implementation

```
def get_guess(player):  
    while True:  
        try:  
            guess = int(input(f"{player}, enter your guess (1-100): "))  
  
            if 1 <= guess <= 100:  
                return guess  
            else:  
                print("Please enter a number between 1 and 100.")  
  
        except ValueError:  
            print("Invalid input! Please enter a number.")  
  
def number_guessing_race():  
    secret_number = random.randint(1, 100)  
  
    print("🎯 NUMBER GUESSING RACE")  
    print("You vs. Computer")  
    print("First one to guess the secret number wins!")  
    print("Guess a number between 1 and 100.\n")
```

Code Highlights

get_guess(player)

Loops until the player enters a valid number between 1 and 100.

Validation

Out-of-range values trigger a prompt; non-numeric input is caught with except ValueError.

number_guessing_race()

Creates the secret number with random.randint(1, 100) and prints the game introduction.

Program Output

```
🎮 NUMBER GUESSING RACE
... You vs. Computer
First one to guess the secret number wins!
Guess a number between 1 and 100.

You, enter your guess (1-100): 98
Your guess is too high.
💻 Computer guessed: 57

You, enter your guess (1-100): 87
Your guess is too low.
💻 Computer guessed: 15

You, enter your guess (1-100): 9
Your guess is too low.
💻 Computer guessed: 8

You, enter your guess (1-100): 8
Your guess is too low.
💻 Computer guessed: 71

You, enter your guess (1-100): 
```

Sample Run

Player Turn

Each guess gets instant feedback: "too high" or "too low".

Computer Turn

The computer makes its own guess right after the player.

Race

Turns alternate until someone guesses the secret number.

Thank You!

Number Guessing Race Project Presentation

S. B. Jain Institute of Technology Management and Research, Nagpur